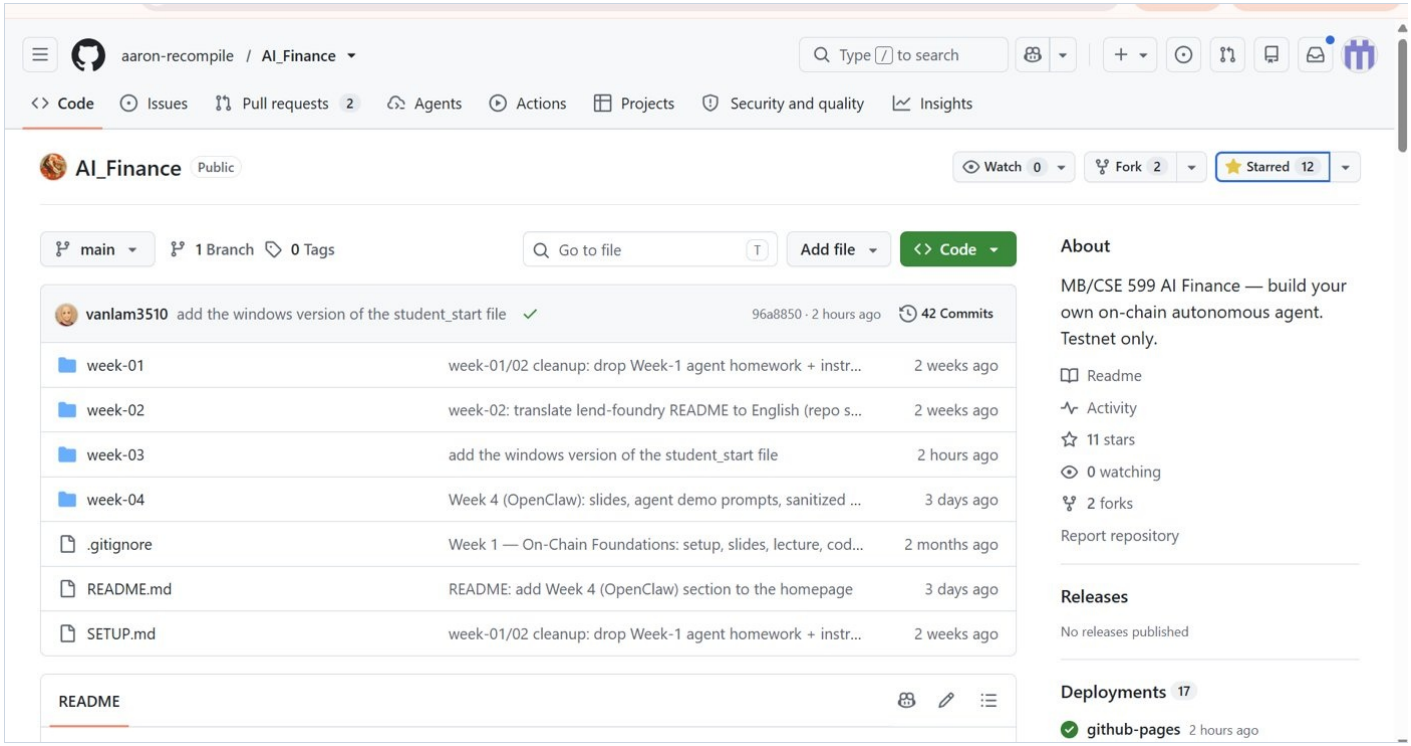
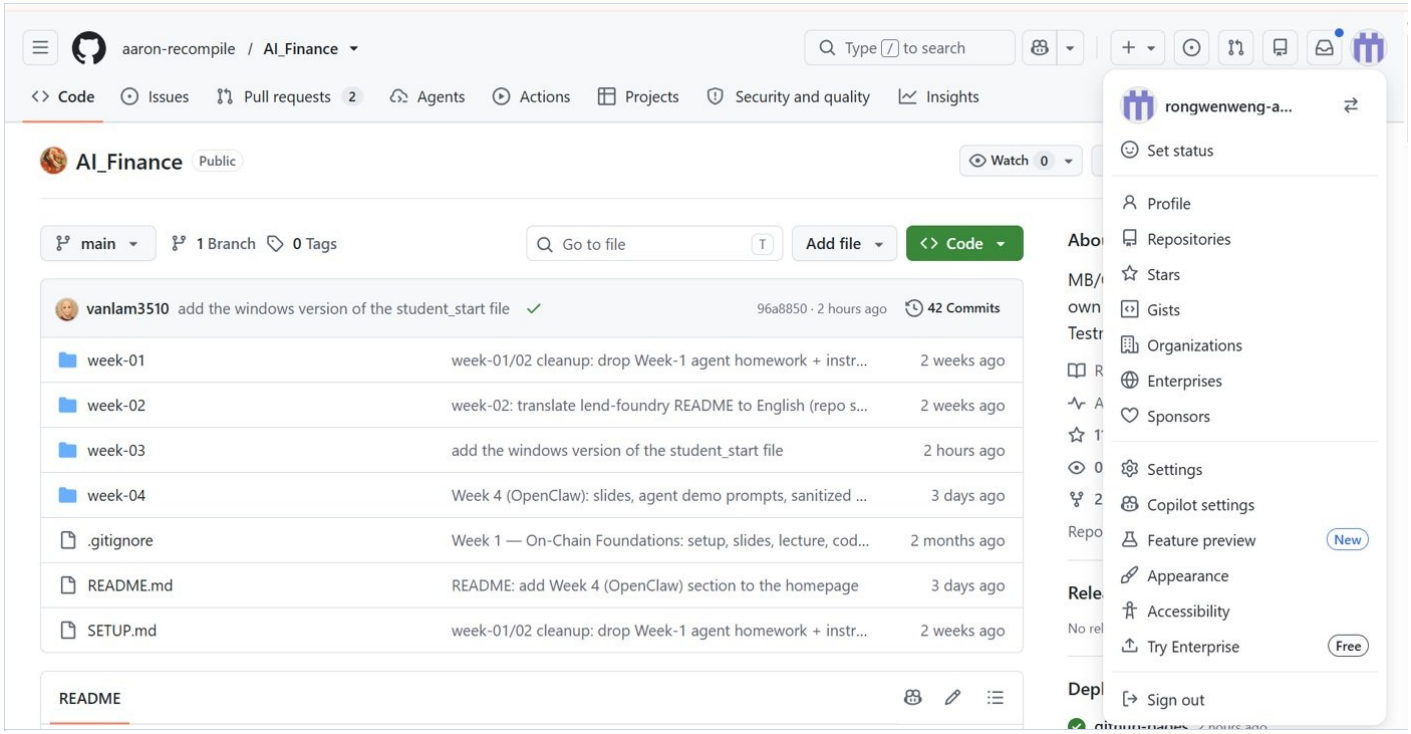
Week 2–3 Homework: DeFi Legos

Student RONGWEN WENG
Course MB/CSE 599 AI Finance
GitHub rongwenweng-arch
Wallet `0x7Bfe6bef0E8092bf2d829a75Af11dDfd6A7b4140` (Base Sepolia)
Bonus PR https://github.com/aaron-recompile/AI_Finance/pull/6

Part 0 — GitHub: course repo starred



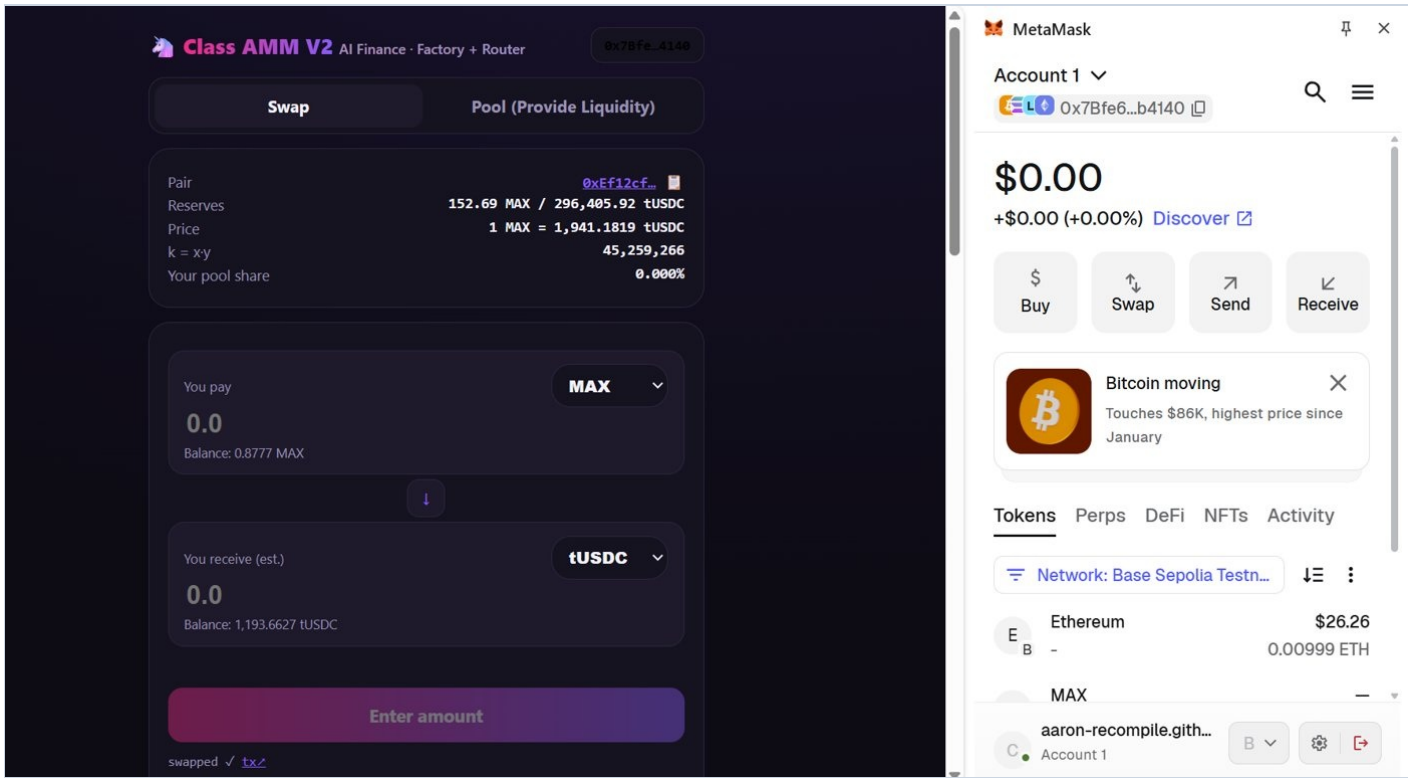
aaron-recompile/AI_Finance with the Star button filled (Starred).



Same page with the account menu open, showing my username rongwenweng-arch.

Part 1.1 — Swap on the AMM (v2.html)

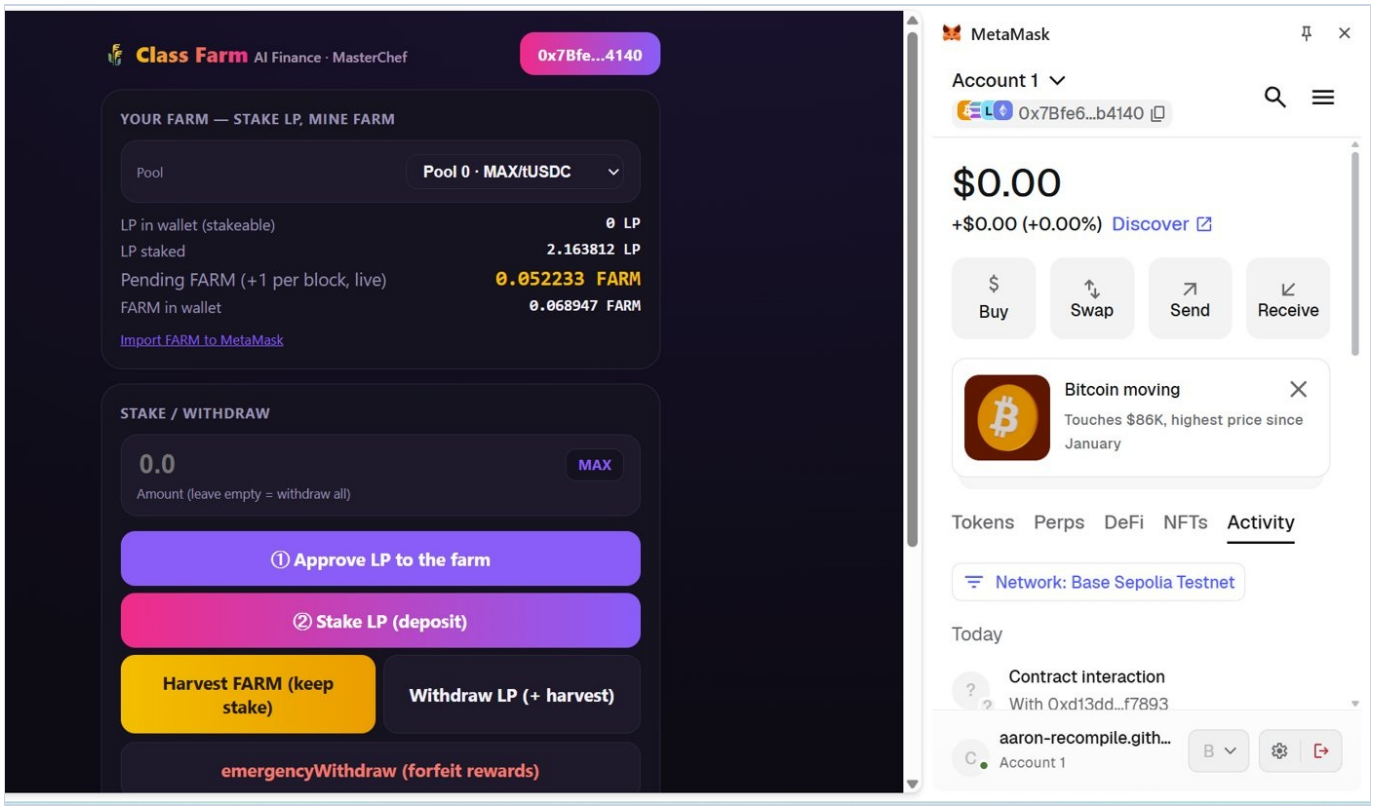
Swapped 0.1 MAX → about 193.66 tUSDC. The pool price moved from 1 MAX = 1,943.72 to 1,941.18 tUSDC (price impact 0.37%). A larger trade moves the price more — that is slippage, because the AMM keeps x·y = k.



Connected 0x7Bfe...4140 · "swapped ✓" · MAX balance 0.9777 → 0.8777, tUSDC 1,000 → 1,193.66 · new price 1,941.18.

Part 1.2 — Farm: stake LP, then harvest (farm.html)

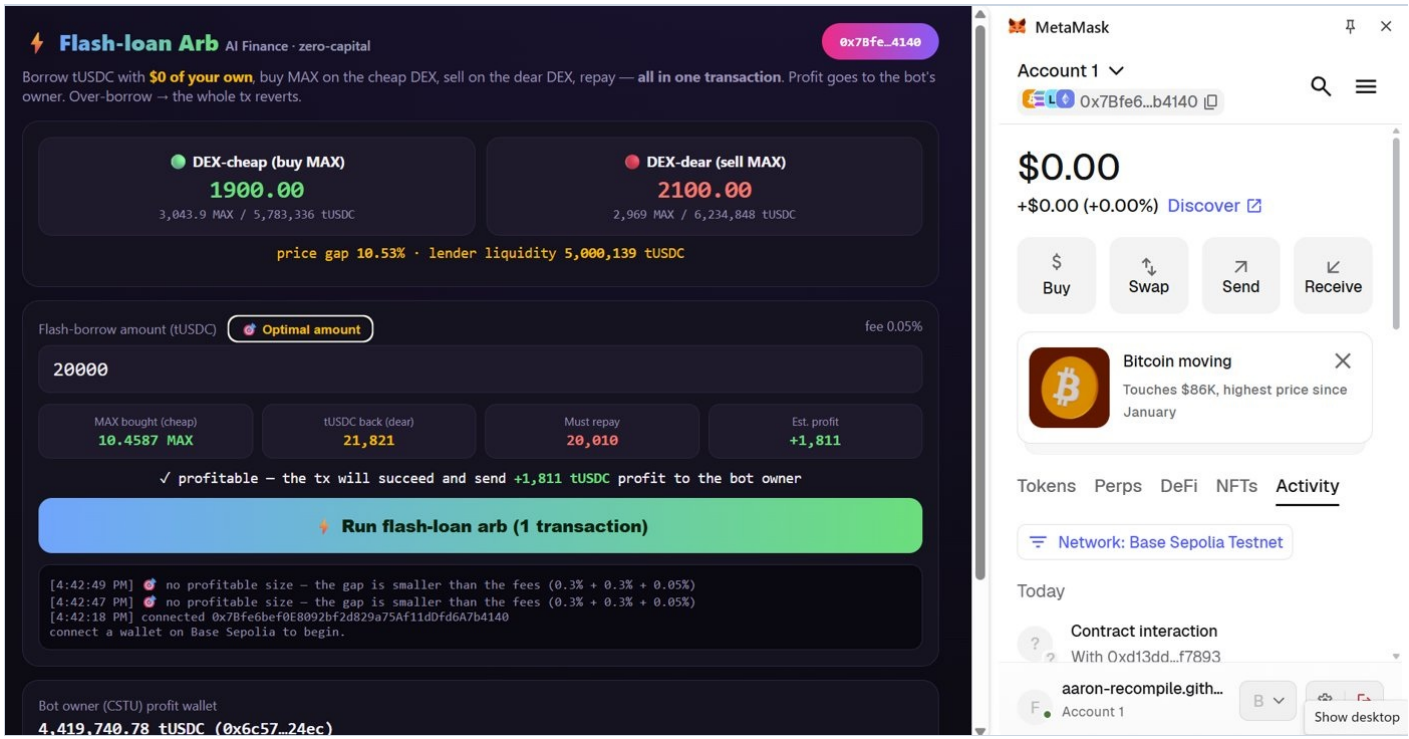
Added 0.05 MAX + 97.06 tUSDC to the MAX/tUSDC pool to receive 2.163812 LP, staked all LP in Pool 0 of the MasterChef, then harvested FARM while keeping the stake.



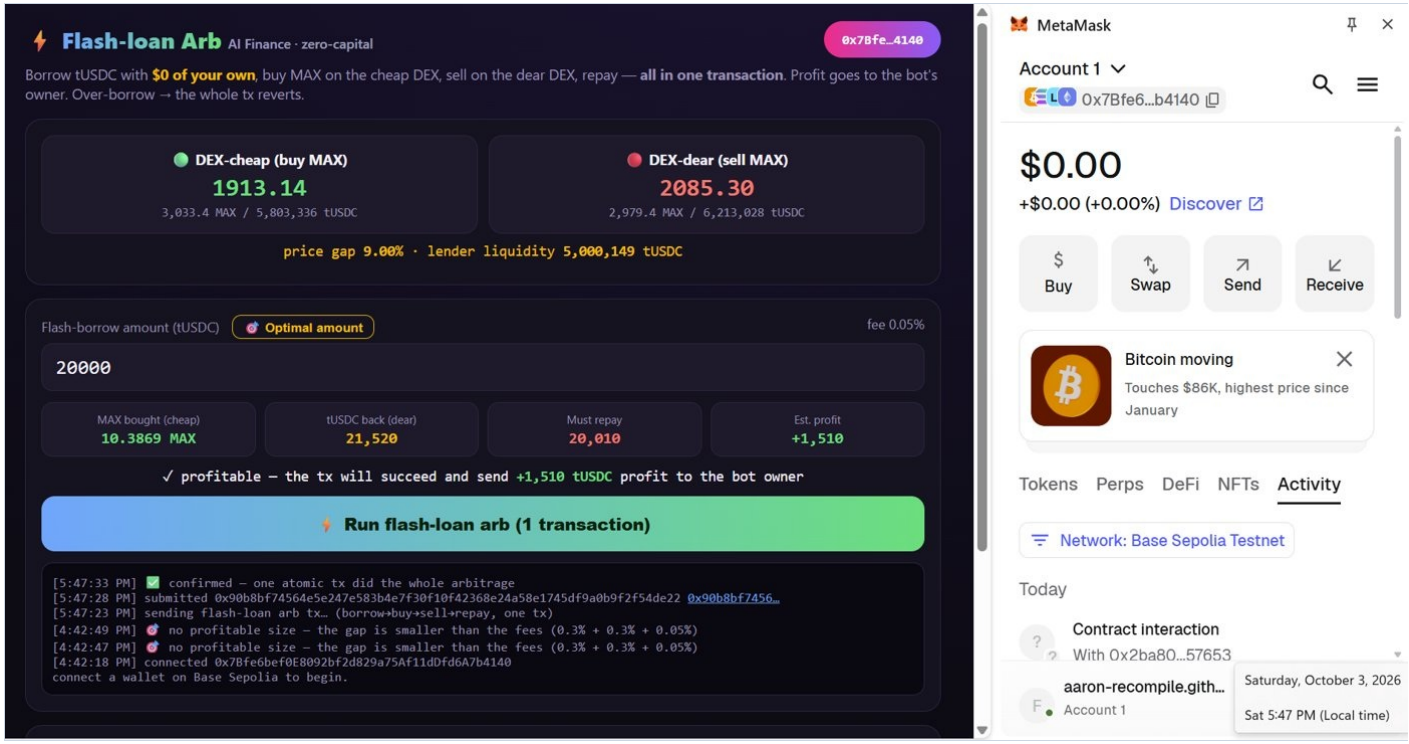
Connected 0x7Bfe...4140 · LP staked 2.163812 · harvested reward: FARM in wallet 0.068947 (pending keeps accruing).

Part 1.3 — Flash-loan arbitrage (flashloan.html)

With a 10.53% price gap, a 20,000 tUSDC flash borrow shows a green profitable estimate. I then ran it: one atomic transaction did borrow → buy on the cheap DEX → sell on the dear DEX → repay. After my trade the gap narrowed from 10.53% to 9.00%.



Connected 0x7Bfe...4140 · green estimate "profitable — +1,811 tUSDC".



Confirmed: "one atomic tx did the whole arbitrage".

Confirmed tx hash:
`0x90b8bf74564e5e247e583b4e7f30f10f42368e24a58e1745df9a0b9f2f54de22`

Part 2 — Bonus: "Pay yourself" Pull Request

PR: github.com/aaron-recompile/AI_Finance/pull/6
File: `week-03/code/flashloan-foundry/src/Arbitrageur.sol`, function `run()`

```
-  usdc.transfer(owner, usdc.balanceOf(address(this)));
+  usdc.transfer(msg.sender, usdc.balanceOf(address(this)));
```

PR description:

> The caller (msg.sender) is the account that triggers the arbitrage and pays the gas, so it should receive the profit, not the deployer stored in owner. Because run() has no access control, on a public chain this means anyone, including MEV bots that copy a pending transaction from the mempool, can call it and take the profit, so the opportunity belongs to whoever gets their transaction included first.

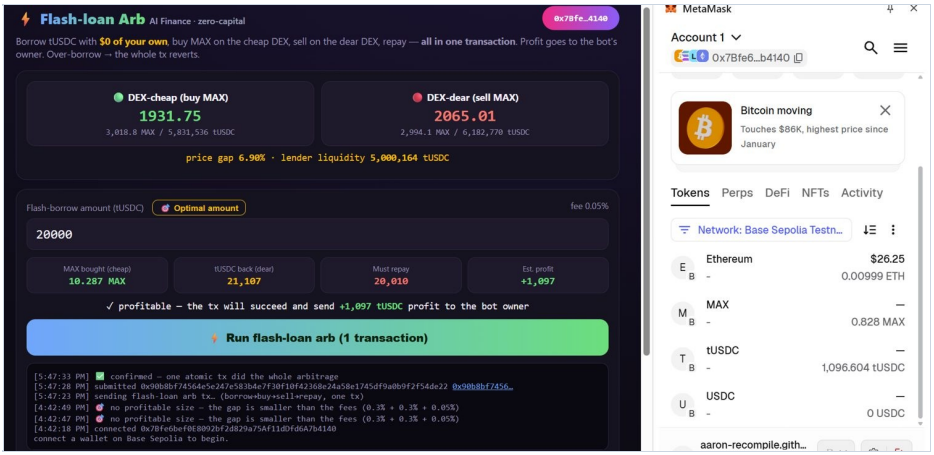
Why it is correct: `owner` is set once in the constructor (`owner = msg.sender`), so it always equals the deployer (the course wallet). Inside `run()`, `msg.sender` is whoever calls the function in that transaction. Swapping one for the other sends the profit to the person who ran the bot. Because anyone can call `run()`, the profit now goes to the first caller, which is exactly what front-running and MEV bots compete for.

Optional Stretch — My own Arbitrageur on Base Sepolia

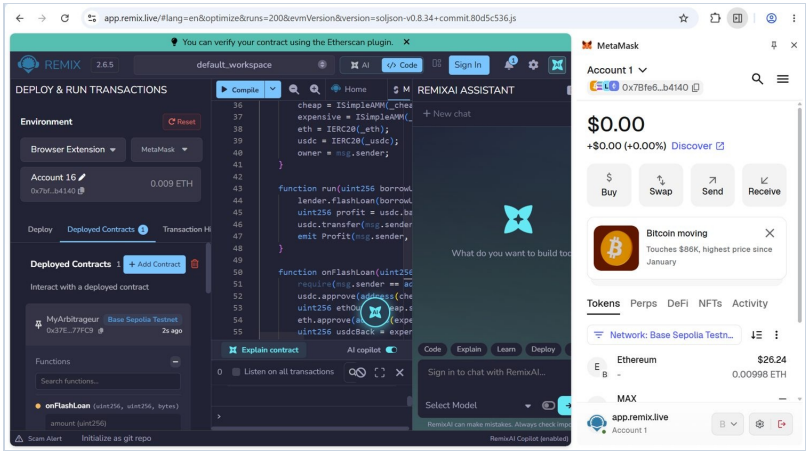
I deployed my own copy of the bot, with the fix from Part 2 (profit to `msg.sender`), using Remix and MetaMask. It reuses the course's existing FlashLender and the two DEX pools, so the only new contract is mine. Then I called `run(20000 tUSDC)` once while the price gap was open (6.90%).

My contract	`0x37E24f9b3B61d7E8108b6474F4E1724D79177FC9`
Deployer and caller	`0x7Bfe6bef0E8092bf2d829a75Af11dDfd6A7b4140` (my wallet)
Constructor	lender 0x1F76…B51A · cheap DEX 0xfE36…d7f9 · dear DEX 0x41c5…CEff · MAX 0x2220…dAa5 · tUSDC 0x6797…Ac69
run() tx	`0x8c635220081945b7be247138a1a2e300041d4c7fedaad804731f2550831e20fa`
Borrowed	20,000 tUSDC (flash loan, repaid in the same tx with the 0.05% fee)

```
function run(uint256 borrowUsdc) external {
    lender.flashLoan(borrowUsdc, address(this), "");
    uint256 profit = usdc.balanceOf(address(this));
    usdc.transfer(msg.sender, profit);   // pay the caller, not owner
    emit Profit(msg.sender, profit);
}
```

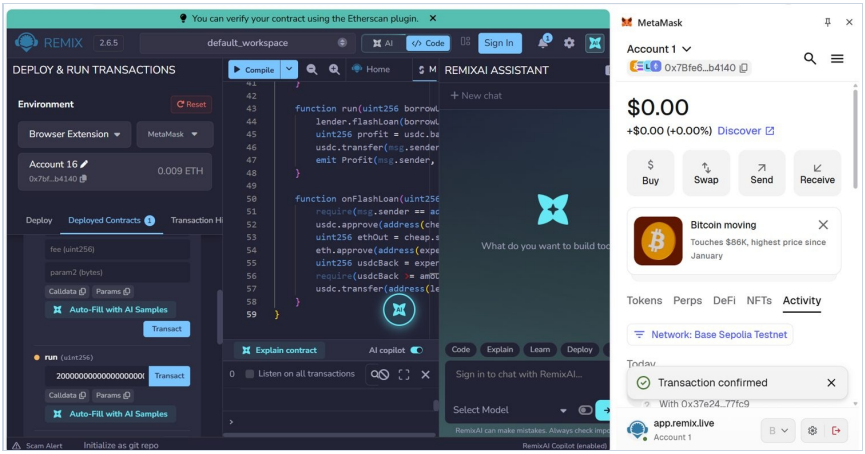


Before: gap 6.90%, estimate +1,097 tUSDC; my wallet holds 1,096.604 tUSDC.

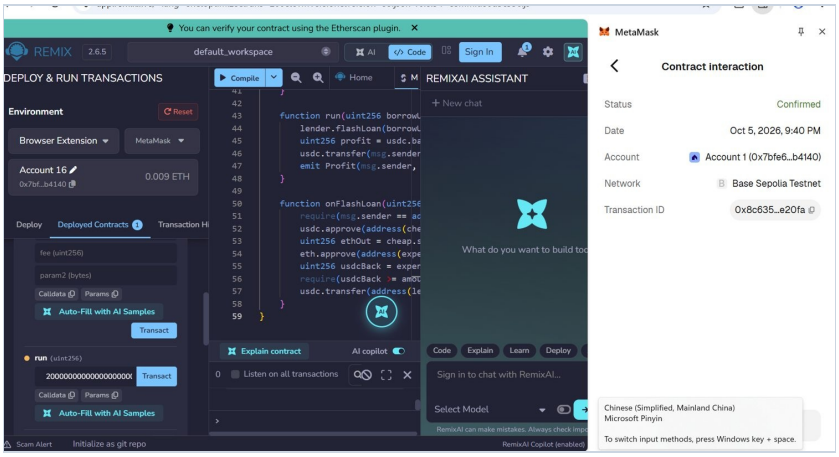


MyArbitrageur deployed on Base Sepolia Testnet at 0x37E…77FC9 from my wallet (Remix + MetaMask).

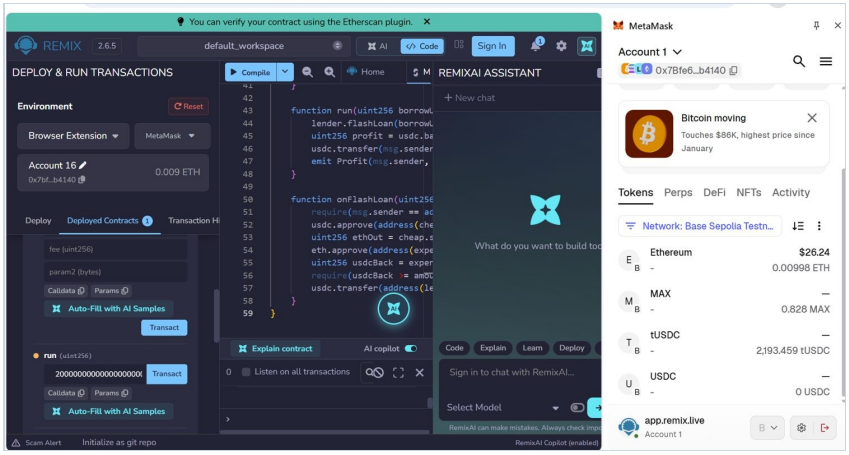
Stretch — Result: my balance went up



run(20000e18) sent from Remix; MetaMask shows "Transaction confirmed" with 0x37e24...77fc9.



Transaction details: Confirmed, Oct 5 2026, Account 0x7bfe6...b4140, Base Sepolia, tx 0x8c635...e20fa.



After: tUSDC balance 2,193.459.

Balance change: 1,096.604 → 2,193.459 tUSDC = **+1,096.855 tUSDC**, matching the page's estimate of +1,097. I used zero capital of my own for the trade and paid only gas.

What this shows: with `owner`, the profit would go to whoever deployed the contract. With `msg.sender`, it goes to whoever calls `run()`. Here I am both, but if anyone else called my contract while the gap was open, they would take the profit. That is the public-chain risk from Part 2: an unprotected `run()` is an open opportunity for anyone, including MEV bots.